Equipe ou agente sozinho:
um teste desenhado para o caso em que a pesquisa diz que a equipe ganha

Gabriel Cortez Claude Code

16 de outubro de 2026

Resumo

No primeiro teste, um terminal sozinho venceu a equipe numa tarefa sequencial. Para saber se o nosso método de orquestração funciona onde a pesquisa diz que equipes ganham, criamos uma tarefa nova: quatro módulos independentes em arquivos separados, quatro defeitos plantados e doze regras sutis, avaliados por 46 testes escondidos. Rodamos três montagens, três vezes cada: o Opus sozinho, uma equipe paralela (Narrador Opus com um dev Sonnet e um dev Haiku) e o Opus escrevendo com um revisor Sonnet de contexto limpo. As nove rodadas tiraram 46 de 46. O agente sozinho foi o mais rápido (1,8 min, contra 2,8 da equipe e 2,3 do revisor) e usou 4 vezes menos tokens que a equipe. A equipe custou o mesmo que o agente sozinho, porque o trabalho foi para modelos baratos. O revisor custou 36% a mais e aprovou as três entregas sem achar nada. O protocolo do NarraTer não falhou nenhuma vez: 0 de 15 pedidos foram dados por prontos antes da hora. A equipe perdeu por três motivos: o dev mais lento (Haiku) ditou o ritmo, o líder somou de 40 a 50 s sozinho, e cada módulo era pequeno demais para compensar a coordenação.

Rótulos. [citado]: afirmado pela fonte indicada. [medido]: medido por nós, com método reproduzível. [inferido]: nossa leitura, sem medição direta.

1 Por que um teste novo

No primeiro teste (relatório *quando-orquestrar.pdf*), um terminal sozinho venceu a equipe de três agentes: mesma nota (31 de 31), 2,5 vezes mais rápido e 7 vezes menos tokens [medido]. Isso bate com a literatura, mas a tarefa daquele teste é justamente a que a literatura diz que não favorece equipes:

- é **sequencial e acoplada**: uma funcionalidade e duas correções no mesmo módulo, com a mesma regra de preço atravessando tudo;
- está **no teto**: o agente sozinho tira 31 de 31, muito acima do limiar de ∼45% a partir do qual coordenar deixa de pagar (Kim et al. [1]) [citado];
- é **pequena**: o Opus sozinho termina em 1,6 min, e não sobra trabalho para dividir.

Com isso, aquele teste só podia mostrar o custo da equipe, nunca o ganho. Para saber se o *nosso método* de orquestração funciona, precisamos de uma tarefa em que a equipe *deveria* ganhar, e ver se ela ganha. Se não ganhar nem aí, a falha é do NarraTer (protocolo, divisão do trabalho, papéis), e não da ideia de orquestrar. É essa resposta que a próxima etapa precisa: corrigir as falhas do método e construir o gerador automático de equipes, que vai decidir quando montar uma.

2 Onde a pesquisa diz que a equipe ganha

Tabela 1: Casos em que a orquestração ganhou na literatura [citado], e como o teste novo cobre cada um.

Caso	O que foi medido	No teste novo
Trabalho paralelizável	Coordenação centralizada: +80,8% em tarefas paralelizáveis [1]. Pesquisa multiagente da Anthropic: +90,2% sobre o Opus sozinho [2]	Quatro módulos independentes, em arquivos separados; a equipe divide os módulos
Revisor com contexto limpo	Revisor que só vê o diff: ∼2 bugs por PR, 58% graves [3]; testes como retorno: +4 a 5 pontos [4]	Montagem "dev + revisor": o revisor não vê o histórico de quem escreveu
Tarefa difícil para o modelo	Único ganho significativo da Vals.ai: +7,3 pontos no modelo com menor nota sozinho [5]	Doze regras sutis, escritas mas fáceis de deixar passar (nível *Sutil*)
Modelos diferentes	Debate com modelos diferentes melhora de forma consistente [6]	Revisor Sonnet revisando o Opus; equipe com Opus, Sonnet e Haiku

Ficou de fora o caso do contexto grande demais para um agente (LangChain [7]): ele pede uma tarefa muito maior que o orçamento de uma rodada.

3 O teste

3.1 A tarefa `armazem`

Um repositório Python com quatro módulos de uma loja online, cada um num arquivo próprio e sem importar os outros:

- **frete**: peso cúbico, com arredondamento para o próximo meio quilo, mais uma tarifa cobrada a mais nos pesos exatos de 1, 5 e 10 kg;
- **cupons**: cupom "leve N, pague M", mais um cupom recusado no último dia de validade;
- **estoque**: depósitos e transferência entre eles, mantendo o custo PEPS dos lotes, mais um custo errado quando a saída passa de um lote para outro;
- **parcelas**: juros pela tabela Price acima de 3 parcelas, mais um erro com a primeira parcela num dia 31.

Cada módulo tem um pedido de funcionalidade (`docs/PEDIDOS.md`), uma reclamação de cliente com um defeito plantado (`docs/RECLAMACOES.md`) e regras em `REGRAS.md`. Algumas regras são fáceis de deixar passar, por exemplo:

- o frete grátis é negado quando o peso *cúbico* passa de 30 kg;
- no "leve N, pague M", linhas do mesmo produto contam juntas e valem pelo menor preço;
- um lote transferido mantém a antiguidade da entrada original no depósito de destino;
- o vencimento das parcelas é sempre calculado a partir da primeira, mesmo quando uma parcela caiu num fim de semana.

A nota vem de **46 testes escondidos** em cinco níveis: um por módulo (funcionalidade, defeito e regressão, de 8 a 10 cada) e *Sutil* (12). A solução de referência passa nos 46; o repositório sem as correções passa em 12.

3.2 Três montagens

Tabela 2: As três montagens. O pedido é o mesmo; muda só a primeira frase, que diz quem faz o quê.

Montagem	Agentes	Instrução
Sozinho	Opus 5.5 (papel de desenvolvedor)	"Você trabalha sozinho"
Equipe paralela	Narrador Opus 5.5 (líder), DevA Sonnet 5.5, DevB Haiku 5.5	Dividir os módulos entre DevA e DevB, sem os dois mexerem no mesmo arquivo; o Narrador coordena, revisa e integra
Dev + revisor	Opus 5.5 escreve; Revisor Sonnet 5.5 só lê	Ao terminar, pedir a revisão dizendo só onde está o repositório; corrigir o que for procedente

O método é o do benchmark de versões (D19):

- cada rodada começa do zero, com HOME, tmux e sessões próprios;
- são 3 rodadas por montagem, em ordem contrabalançada, na versão atual do NarraTer (H, `73fab8b`);
- a tarefa termina quando o agente que recebeu o pedido escreve a linha de fim, ou após 40 min.

Comando:
`BENCH_TAREFA=armazem BENCH_VARIANTE=solo|equipe|revisor python3 rodar.py H N`.

3.3 Previsões, registradas antes de rodar

Escritas a partir da literatura, antes da primeira rodada válida:

P1 **Tempo:** a equipe paralela termina antes do agente sozinho, porque os módulos são independentes (Kim et al.: +80,8% em tarefas paralelizáveis).

P2 **Custo:** a equipe paralela gasta mais tokens e mais dinheiro que o agente sozinho (Vals.ai: de 1,8 a 5,1 vezes; Anthropic: ∼15 vezes os tokens de uma conversa).

P3 **Qualidade com revisor:** a montagem "dev + revisor" passa em pelo menos tantos testes escondidos quanto o agente sozinho, com o ganho no nível *Sutil*, a um custo de 1,3 a 2 vezes (Cognition, MetaGPT).

P4 **Teto:** se o agente sozinho passar em quase todos os 46, nenhuma montagem melhora a nota (Kim et al.: acima de ∼45%, coordenar não paga em qualidade).

P5 **Amplificação de erro:** se a equipe paralela errar mais que o agente sozinho, o erro estará nos módulos do DevB (Haiku), e o Narrador não o terá pegado na revisão (MAST: verificação fraca).

4 Resultados

Tabela 3: As três montagens na tarefa `armazem` [medido]. Média e, entre parênteses, o mínimo e o máximo das 3 rodadas. Dados: `bench/orquestracao/resultados/2026-10-16/`, tabela gerada por `armazem.py`.

	Sozinho	Equipe paralela	Dev + revisor
Testes escondidos (de 46)	46 (46–46)	46 (46–46)	46 (46–46)
dos quais Sutil (de 12)	12 (12–12)	12 (12–12)	12 (12–12)
Duração (min)	1,8 (1,7–1,9)	2,8 (2,6–3,0)	2,3 (2,2–2,4)
Tokens processados (milhões)	0,36 (0,29–0,40)	1,51 (1,27–1,87)	0,58 (0,58–0,59)
Custo de API equivalente (US$)	0,56 (0,54–0,58)	0,60 (0,47–0,78)	0,76 (0,74–0,77)
Custo por teste aprovado (centavos)	1,2 (1,2–1,3)	1,3 (1,0–1,7)	1,6 (1,6–1,7)
Chamadas à API (todos os agentes)	8 (7–9)	31 (27–38)	13 (13–14)
Pedidos a outros agentes	0 (0–0)	2 (2–3)	1 (1–1)
Linhas no diff	372 (344–396)	399 (357–439)	352 (338–379)

- **Qualidade igual, no teto.** As nove rodadas passaram nos 46 testes escondidos, inclusive nos 12 sutis. O Opus sozinho não deixou passar nenhuma regra [medido].
- **O agente sozinho foi o mais rápido.** Levou 1,8 min, contra 2,8 da equipe paralela e 2,3 da montagem com revisor, sem sobreposição nas duas comparações (Mann–Whitney $U = 0$, $p = 0{,}10$, o menor p possível com 3 contra 3) [medido].
- **Tokens:** a equipe processou 4,2 vezes os tokens do agente sozinho (1,51 contra 0,36 milhão), e o revisor 1,6 vez, os dois sem sobreposição [medido].
- **Custo:**
  - A equipe custou o mesmo que o agente sozinho (US$ 0,60 contra 0,56; as rodadas se sobrepõem, $p = 1{,}0$). Os tokens a mais foram quase todos do Haiku, a cerca de US$ 0,03 por rodada.
  - A montagem com revisor custou 36% a mais (US$ 0,76), sem sobreposição. O revisor custou de US$ 0,14 a 0,17 por rodada [medido].
- **O protocolo funcionou.** Nenhum dos 15 pedidos entre agentes foi dado por pronto antes da hora, nenhuma resposta veio cortada, e o fim do turno foi detectado em 0,2 a 2,9 s. As falhas que corrigimos nas versões E a G (D20) não voltaram [medido].

4.1 Por que a equipe paralela foi mais lenta

Tabela 4: Onde foi o tempo da equipe paralela [medido], em segundos. As colunas dos devs são o tempo em que cada um estava trabalhando.

Rodada	Duração	Só o Narrador	DevA (Sonnet)	DevB (Haiku)	Os dois juntos
1	181	49	56	127	51
2	167	47	59	120	59
3	157	39	49	109	39

1. **O dev mais lento dita o ritmo.** Nas três rodadas, o Narrador deu dois módulos a cada dev. O Haiku levou de 109 a 127 s para os dele, o dobro do Sonnet, e os dois trabalharam ao mesmo tempo só de 39 a 59 s [medido]. Numa equipe paralela, a duração é a do membro mais lento, e misturar modelos de velocidades diferentes desperdiça o paralelismo [inferido].
2. **O líder custa um tempo fixo.** O Narrador ficou sozinho de 39 a 49 s por rodada: lendo os documentos, dividindo o trabalho e conferindo no fim. Isso é quase metade do tempo que o Opus sozinho levou para a tarefa inteira (102 a 114 s) [medido].

3. **Os pedaços eram pequenos.** O Opus resolve os quatro módulos em menos de 2 min, cerca de 25 s por módulo. Dividir só compensa quando cada parte custa bem mais que a coordenação, que aqui passou de 1 min [inferido].

4.2 O revisor

O revisor foi chamado uma vez por rodada, trabalhou de 26 a 29 s e aprovou as três entregas. Nas três, ele confirmou as quatro reclamações e conferiu os quatro pedidos contra as regras [observado]. Não havia o que achar: o Opus já entregava 46 de 46. Pela pesquisa, o revisor rende quando quem escreve erra; aqui ele só acrescentou 0,5 min e 36% de custo [inferido]. Numa rodada, ele tentou responder com uma ferramenta interna do Claude (`SendMessage`), que falhou; a resposta chegou mesmo assim, porque o NarraTer a lê no transcript (D20) [observado].

5 As previsões contra o medido

Previsão	O que aconteceu	Veredito
P1, a equipe paralela é mais rápida	2,8 contra 1,8 min, sem sobreposição	**Refutada**
P2, a equipe gasta mais	4,2 vezes os tokens; custo igual (modelos baratos)	Em parte
P3, o revisor melhora a nota	Nota igual (sem espaço), 36% mais caro, aprovou tudo	Não testável aqui
P4, no teto, ninguém melhora a nota	46 de 46 nas nove rodadas	**Confirmada**
P5, os erros da equipe vêm do Haiku	A equipe não errou; o Haiku foi o gargalo de tempo	Não se aplica

A refutação de P1 é o resultado mais importante. A pesquisa (Kim et al.) diz que coordenação centralizada ganha em tarefas paralelizáveis, e esta tarefa era paralelizável, mas a equipe do NarraTer perdeu. A causa não está no protocolo, que funcionou sem falhas, e sim na forma da equipe e no tamanho das partes. É aí que o método precisa de correção [inferido].

6 O que isso muda na próxima etapa

[inferido] Para a correção das falhas e para o gerador de equipes:

1. **O portão da D22 se confirma, e fica mais preciso.** Ser paralelizável não basta. Cada parte precisa valer bem mais que o custo fixo de coordenar, que medimos em cerca de 1 min de líder. Para tarefas que o agente sozinho termina em poucos minutos, a resposta é "sozinho".
2. **Em equipes paralelas, modelos de mesma velocidade.** Misturar Sonnet e Haiku para escrever em paralelo desperdiça o paralelismo. A mistura de modelos rende mais na revisão que na escrita.
3. **O revisor só vale quando quem escreve erra.** Com o Opus escrevendo, ele só custou. O caso promissor, sugerido pela Vals.ai (o ganho apareceu no modelo mais fraco), é o inverso: um modelo barato escrevendo e um revisor forte. Se "Sonnet + revisor" chegar à nota do Opus sozinho por menos, esse vira o modelo de equipe padrão do gerador.
4. **O líder deve sair do caminho.** De 39 a 49 s de cada rodada da equipe foram do Narrador sozinho. A lista de tarefas com pega automática (O8) e o comando de aceite rodado pelo backend (O10) atacam exatamente esse tempo.
5. **O benchmark ainda não tem uma tarefa difícil.** Duas tarefas e 18 rodadas depois, o Opus nunca deixou de passar um teste. Para medir ganho de qualidade, o próximo teste precisa de uma tarefa em que o agente sozinho erre, ou de um modelo que escreva pior.

7 Limites

- São 3 rodadas por montagem, então $p = 0{,}10$ é o máximo de evidência possível; afirmamos diferença só onde as rodadas não se sobrepõem.
- A tarefa voltou a ficar no teto para o Opus, apesar das regras sutis. O teste não mede ganho de qualidade.
- A equipe usa o mesmo trio do estudo de versões (Opus, Sonnet, Haiku), para comparar com ele. Uma equipe com modelos iguais pode se sair melhor no tempo; isso não foi medido.
- Os devs trabalharam na mesma pasta, em arquivos separados (o pedido proíbe mexer no mesmo arquivo), e não em worktrees.
- As rodadas foram feitas em sequência, num só dia, na ordem sozinho, equipe, revisor, equipe, revisor, sozinho, revisor, sozinho, equipe.

Referências

[1] Y. Kim et al. *Towards a Science of Scaling Agent Systems*. arXiv:2512.08296, 2025. `https://arxiv.org/abs/2512.08296`

[2] Anthropic. *How we built our multi-agent research system*, 2025. `https://www.anthropic.com/engineering/multi-agent-research-system`

[3] W. Yan (Cognition). *Multi-Agents: What's Actually Working*, 2026-04-22. `https://cognition.com/blog/multi-agents-working`

[4] S. Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. arXiv:2308.00352, 2023. `https://arxiv.org/abs/2308.00352`

[5] Vals.ai. *Do Agent Teams Pay Off? A Case Study on Vibe Code Bench*, 2026-10-09. `https://www.vals.ai/blogs/multi-agent-vibe-code-bench`

[6] H. Zhang et al. *Stop Overvaluing Multi-Agent Debate: We Must Rethink Evaluation and Embrace Model Heterogeneity*. arXiv:2502.08788, 2025. `https://arxiv.org/abs/2502.08788`

[7] LangChain. *Benchmarking Multi-Agent Architectures*, 2025. `https://www.langchain.com/blog/benchmarking-multi-agent-architectures`